UNIVERSIDAD DE LAS AMÉRICAS (UDLA)

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ISWZ1102 – PROGRAMACIÓN I

EJERCICIO DE PROGRAMACIÓN – S4

Nombre: Kathleen Bustos Jácome

Fecha: viernes, 16 de octubre de 2026

Descripción del Problema: Una tienda desea implementar un programa en C que permita gestionar su inventario de productos. El programa debe recibir los nombres de los productos y sus precios, para luego realizar varias operaciones, tales como:

a) ANTES DE EMPEZAR A PROGRAMAR

- **¿Cómo manejarías la relación entre los nombres y los precios de los productos?**

Se maneja mediante el uso de arreglos paralelos. La relación lógica entre el nombre de un producto y su respectivo valor comercial se mantiene a través del índice filas que ocupan en sus respectivos arreglos. Si un producto X se almacena en productos[filas], su valor correspondiente estará ubicado en precio[filas].

- **¿Cómo podrías hacerlo si no puedes usar estructuras ni punteros?**

Al no utilizar estructuras (struct) ni punteros explícitos, la alternativa adecuada es declarar dos arreglos separados:

Una matriz bidimensional de caracteres char productos[MAX][50] para almacenar las cadenas de texto y un arreglo unidimensional float precio[MAX] para los precios.

b) ALTERNATIVAS DE SOLUCIÓN

- **¿Podría existir más de una alternativa de solución al problema?**

Sí, en programación siempre existen múltiples alternativas para resolver un mismo problema, ya que depende mucho de la lógica, el estilo y los conocimientos de cada programador. En este caso, una alternativa consistiría en escribir todo el programa directamente dentro del main() de manera secuencial, mientras que existiría otra alternativa que plantee estructurar el programa mediante funciones modulares en un solo archivo .c

- **¿Qué alternativa podría ser más eficiente o adecuada para este problema?**

En este caso opté por la segunda alternativa porque organizar el código en funciones modulares mantiene la función main() corta y limpia, evita repetir la lógica de cálculo y hace que el sistema sea mucho más fácil de revisar o ajustar.

c) ESTRUCTURA Y MÓDULOS DEL PROGRAMA

El sistema desarrollado maneja hasta un máximo de 10 productos definido en una variable global (#define MAX 10) y se compone de las siguientes funciones principales:

Función	**Parámetros que recibe**	**Tipo de Retorno**	**Descripción / Objetivo**
ingresarproducto	productos[ ][50], precio[ ], cantidad	int	Gestiona el ingreso del nombre y precio del producto en la posición actual del arreglo, retornando el contador actualizado de productos.
calculopreciottotal	precio[ ], cantidad	float	Recorre los elementos registrados y retorna la suma acumulada de los precios.
obtenerbaratoycaro	productos[ ][50], precio[ ], cantidad	void	Evalúa mediante un bucle de comparación los valores máximo y mínimo del inventario, guardando sus nombres mediante la función strcpy de la librería <string.h>.
mostrarpreciodeproducto	cantidad, productos[ ][50], precio[ ]	void	Recibe un nombre a buscar y utiliza la función strcmp para realizar la búsqueda exacta e imprimir el precio asociado.

d) VARIABLES UTILIZADAS

- **MAX:** Define la capacidad máxima de productos en el inventario (10).
- **productos[MAX][50]:** Matriz para guardar los nombres de los productos.
- **precio[MAX]:** Arreglo para almacenar los precios de cada producto.
- **cantidad:** Contador del número de productos ingresados.
- **op:** Guarda la opción elegida por el usuario en el menú.
- **filas:** Índice para recorrer los arreglos en los bucles for.
- **preciottotal:** Almacena la suma total de los precios.
- **preciopromedio:** Guarda el promedio calculado de los precios.

- **preciomayor / preciomenor:** Almacenan el precio más alto y el más bajo.
- **productoconpreciomayor / productoconpreciomenor:** Guardan el nombre del producto más caro y del más barato.
- **buscarproducto:** Almacena el nombre del producto a consultar.
- **encontrado:** Indicio o bandera (0 o 1) que señala si el producto buscado existe.

e) CONCLUSIONES

El uso de arreglos paralelos (productos y precio) permitió vincular los nombres y precios de los productos manteniendo la relación de datos sin recurrir a estructuras ni punteros explícitos.

La creación de módulos mediante funciones independientes simplificó la estructura del programa principal (main), mejoró la legibilidad del código y permitió la reutilización de funciones de cálculo como calculopreciotot al.

La inclusión de validaciones de control, como el límite de capacidad (MAX) y la verificación de registros previos antes de calcular el promedio o realizar comparaciones, garantiza un funcionamiento correcto y evita errores de ejecución.